

Fully Open-Source Accelerator for Unified and Protected ML-KEM-1024 and ML-DSA-44

Dillibabu Shanmugam and Patrick Schaumont
Worcester Polytechnic Institute, Worcester, MA, USA
{dshanmugam, pschaumont}@wpi.edu

Abstract—Coproducts accelerate post-quantum cryptography, and leakage assessment lags behind. Because a unified coprocessor exposes its arithmetic as an ISA, every custom instruction that touches a secret needs a leakage assessment before deployment. We assess in two domains. The pre-silicon domain estimates leakage from per-cycle toggle counts of an RTL simulation while the design can still change, and the post-silicon domain measures leakage in power traces of a ChipWhisperer CW305. One RTL base serves ML-KEM-1024 and ML-DSA-44, unprotected and protected, and CV-X-IF issues the custom instructions. Yosys, nextpnr-xilinx and Project X-Ray produce the bitstream, and no vendor tool or proprietary IP block enters the flow. The ISA holds 62 custom instructions, and the assessment covers the ten selected instructions that carry a secret. A fixed-versus-random test at 500 runs per class detects secret dependence on the six single-operand instructions and public-operand dependence under a constant secret on the four others, in both domains. The two domains return the same verdict on all ten, and their activity correlates at a mean of 0.824. The assessment quantifies leakage per instruction before board measurement, a designer acts on a measured quantity instead of a pass or fail verdict, and the design, flow, traces and analysis are released.

Index Terms—post-quantum cryptography, ML-KEM, ML-DSA, side-channel leakage assessment, pre-silicon, open-source FPGA flow, RISC-V custom instructions

I. INTRODUCTION

NIST finalized ML-KEM and ML-DSA in August 2024 [1], [2], and the draft transition plan deprecates the 112-bit-security parameter sets after 2030 and disallows quantum-vulnerable ones after 2035 [3]. PQC resists a quantum adversary, but its implementations must also resist timing and power side channels. Timing effects count in clock cycles, which RTL verification covers. Power effects arise below the cycle, where glitches, routing and coupling between nets cause leakage [4], and their measurement needs a built prototype.

Reproducibility is the challenge. A prototype for power assessment of PQC covers source code, tool flow, target architecture, measurement setup and trace processing, and every abstraction level modulates the side channel, from memory layout and register allocation to forwarding, netlist and routing [5]. Reproducibility is a must for the new standards, because the assessment methodology is not yet standard and the standards specify functionality rather than secure implementation. ISO/IEC 17825:2024 defines no test class for lattice schemes [6], and its fixed-key against random-key split suits them poorly, because public and private lattice material are

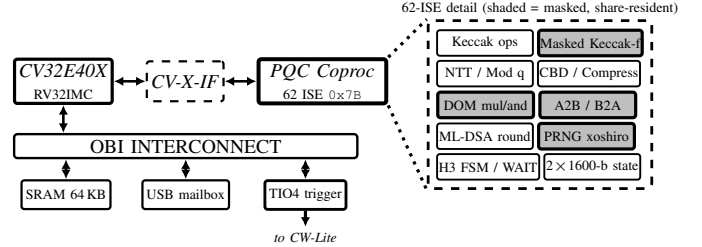


Fig. 1. The platform. A CV32E40X core issues 62 custom instructions to the PQC coprocessor over CV-X-IF, with SRAM, a host mailbox and a TIO4 trigger on the OBI fabric. Every operand crosses one 32-bit port, and the shaded groups keep shares and randomness inside the coprocessor.

TABLE I
COVERAGE OF PRIOR OPEN-SOURCE PQC HARDWARE AND PRE-SILICON TOOLING. THIS WORK COMBINES AN OPEN BITSTREAM PATH WITH A PRE-SILICON AND A POST-SILICON ASSESSMENT OF THE SAME DESIGN.

Work	KEM/SIG	Masking	Open flow	Pre-silicon	Silicon
HORCRUX [10]	both	—	—	—	—
HOPE-MLKEM [11]	KEM	high-order	—	—	FPGA
Two Birds [12]	both	1st-order	—	—	FPGA
ARCHER [13]	—	—	—	arch.	—
SPARC [14]	KEM (SW)	—	—	RTL	—
This work	both	1st-order DOM	yes	RTL	FPGA

linked [7]. A result is reproducible across laboratories only when every layer is documented and visible, yet published assessments run on black-box components and non-disclosure agreements forbid verdicts on proprietary silicon. An open implementation and flow make the whole experiment portable, and that is the precondition for refining the test criteria.

A leakage assessment helps most before silicon exists, because the design can still change. Saidoyoki built both halves of one design and showed that pre-silicon tools flag leaks which are unexploitable in practice [8], and Telescope adds a hierarchy from architecture to gate level along which a leak can be followed [9]. A comparison of gate-level simulation with a measured ASIC concluded that verification requires a complete system model [5]. This paper carries that comparison to a custom-instruction coprocessor, where one instruction is the natural unit of assessment. The 62 instructions differ in secret exposure, and we simulate and measure ten selected secret-relevant instructions on identical operands.

Table I places this work among open-source PQC hardware and pre-silicon tooling. HORCRUX is the closest architectural

relative, a lightweight RISC-V extension for both standards without masking or an open bitstream path [10]. HOPE-MLKEM and the unified masked design of Dobias et al. are masked and measured on FPGA as complete implementations [11], [12], while ARCHER attributes leakage to instructions of a fixed base ISA and VeriSide and SPARC assess leakage at RTL [13]–[15]. Fritzmann et al. measure masked lattice instructions on a RISC-V core with a t -test per instruction [16], and formal verifiers such as COCO check masked software against a netlist [17]. This work follows each custom instruction through both domains on identical operands, and Section III follows on FPGA the vendor-free path that Basilisk documented for an SoC [18].

This paper makes three contributions. (i) A unified 62-instruction CV-X-IF coprocessor for ML-KEM-1024 and ML-DSA-44, unprotected and first-order masked from one RTL base, with the mask generator and the domain-oriented gadgets inside the coprocessor. (ii) An open-source implementation flow from SystemVerilog to a configured XC7A100T on a ChipWhisperer CW305 without any vendor tool, released with the design, traces and analysis¹. (iii) A pre-silicon and post-silicon leakage assessment of ten selected secret-relevant instructions on identical operands, with a quantitative comparison of the two domains.

Section II states the threat model and scope. Section III presents the coprocessor, the CV-X-IF protocol and the open implementation flow. Section IV selects the ten instructions and defines both setups and their comparison. Section V gives the results, performance, area and limits.

II. THREAT MODEL AND SCOPE

The adversary measures the power of the FPGA over a shunt resistor with ChipWhisperer-class equipment and chooses the inputs. Because design and firmware are public, the adversary knows which custom instruction executes in each window. Acquisition is synchronous with the target clock, which reduces misalignment and favours the adversary. The target is first-order leakage of every secret-touching custom instruction, unprotected and masked.

Probing, fault injection, electromagnetic emanation and higher-order analysis lie outside scope, although the open reference supports them. Each of the ten instructions runs in isolation under its own trigger, and the trace counts are isolated measurements rather than bounds on an attack inside a running decapsulation or signature, whose instruction sequence can leak through retained intermediates or reused randomness even when each instruction passes alone. The evaluator also holds the operand and the mask of every trace, which serve profiling and diagnosis, while the adversary holds the public RTL and neither of these.

III. ARCHITECTURE AND OPEN-SOURCE IMPLEMENTATION

A. Unified coprocessor

Fig. 1 shows the system on chip that hosts the coprocessor. A CV32E40X RV32IMC core [19] connects over CV-X-IF to the coprocessor, and an OBI fabric links 64 KB SRAM, a mailbox read by the CW305 USB controller and a register that raises the TIO4 pin, through which C code sets the measurement window. One datapath serves ML-KEM-1024 modulo $q_K = 3329$ and ML-DSA-44 modulo $q_D = 8380417$, because both standards need prime reduction, a transform, noise sampling and coefficient packing. The two share the Keccak state and permutation, the mask generator and the arithmetic-to-Boolean and Boolean-to-arithmetic converters, while each modulus keeps its own reduction tree and a mode bit sets the datapath width. Unification is what makes one assessment cover both standards, since a leak found in a shared unit applies to both.

The coprocessor executes 62 custom instructions in eight groups: 13 for Keccak, 5 for modular arithmetic, 4 for the NTT butterfly, 12 for centred-binomial sampling, 6 for compression and rejection, 2 for Power2Round and Decompose, 18 for masking and share conversion, and 2 for the generator. It attaches to the core under the custom-3 opcode $0x7B$, which RV32IMC leaves unassigned, and Fig. 2 traces one instruction along that path. The core transmits the instruction word with $rs1$ and $rs2$ over the issue channel, and the coprocessor decodes `funct3` and `funct7`, accepts one of the 62 instructions and rejects any other, whereupon the core raises the illegal-instruction exception. Execution follows a three-cycle sequence of latch, evaluate and return through one 32-bit port, whereas result-valid stays low for about 673 cycles during the Keccak permutation and 25 cycles during a share conversion. The adapter omits speculation and kill, and with in-order issue the commit channel is tied off and result-ready is held high. This interface is the boundary the assessment watches, because every operand and every result crosses it in the clear unless the instruction is masked.

One RTL serves both firmware variants, unprotected and protected, and the two differ in the instructions the firmware issues and in nothing else. Secret-touching gadgets use first-order domain-oriented masking [20], and linear kernels process data share-wise. The generator is `xoshiro128++`, a non-cryptographic PRNG that serves as mask stimulus in this study, and mask-consuming instructions stall until all four seed words are written. A deployment needs a cryptographic generator and a check of the seeded state, which this module does not perform. Both firmware images pass their FIPS known-answer tests on the board. A synthesis-time define selects the hardware masked Keccak, and the measured build excludes it for the reason given in Section III-B. None of the ten instructions uses the hardware masked Keccak, and its exclusion changes no measured instruction.

¹<https://github.com/DillibabuShanmugam/pqc-ise-leakage-fpt2026>

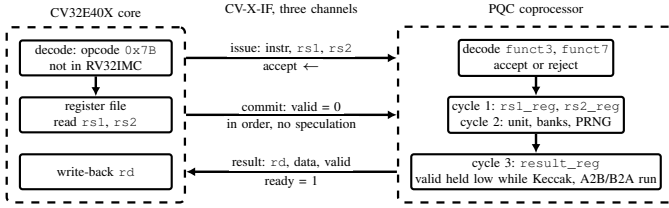


Fig. 2. Handling of a custom opcode over CV-X-IF. The core offers an instruction it cannot decode on the CV-X-IF issue channel, the coprocessor accepts it by `funct3` and `funct7`, and three cycles later the result channel returns `rd`. The commit channel is tied off and result-ready is held high, because the adapter omits speculation and the core issues in order.

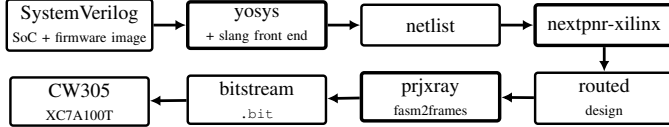


Fig. 3. The bitstream path from SystemVerilog to the configured device, with no vendor tool between them. The firmware image enters at synthesis with the RTL, and every bitstream carries its own operand tables.

B. Open-source implementation flow

Fig. 3 shows the SystemVerilog-to-bitstream path: Yosys 0.50, nextpnr-xilinx with a locally built XC7A100T chip database, Project X-Ray, Verilator 5.037 for simulation, and ChipWhisperer 5.6.1, whose CW305 driver loads the bitstream. The RTL, flow, firmware, traces and analysis scripts are released under the MIT licence. Open tooling makes every intermediate file readable, and an evaluator can trace a leak measured on the board to the netlist element and the instruction that caused it. Because binary-only releases hinder reproducibility, we state each tool obstacle and its fix. Yosys synthesizes the design with its standard commands, but its default front end cannot elaborate the package-typed ports of the masked Keccak core, and the slang front end therefore processes that core separately before the two netlists are linked. The `-nowidelut` flag raises LUT use slightly, while it keeps the Keccak row multiplexers from stalling the placer.

nextpnr-xilinx places and routes the design [21]. With the full masked Keccak the placer fails at 89,560 cells, whereas the design without it holds 61,857 placeable cells and routes at the first attempt. The design carries no timing constraint file, and the 19.63 MHz that nextpnr reports exceeds the 16 MHz board clock. `fasm2frames` expands the FASM output against the Project X-Ray database [22], and `xc7frames2bit` generates the bitstream. The firmware image enters at synthesis, which gives every bitstream its own operand tables and ties each measurement to one build, and one build carries the blocks of all ten instructions. The flow extends the open reference of our OSCAR work [23] from the XC7A35T to the XC7A100T, and Table II gives the resource use of the measured build.

TABLE II
RESOURCE USE OF THE OPEN-FLOW BUILD ON THE XC7A100T. COUNTS ARE NEXTPNR LUT HALVES AND DO NOT COMPARE WITH A VENDOR LUT6 COUNT.

Resource	available	this build	occupancy
SLICE_LUTX	126,800	68,152	53.7 %
SLICE_FFX	126,800	13,917	11.0 %
DSP48E1	240	54	22.5 %
RAMB36E1	135	20	14.8 %

TABLE III
THE TEN INSTRUCTIONS THAT CARRY A SECRET, OF THE 62 THE INTERFACE OFFERS. FW GIVES THE FIRMWARE THAT ISSUES THE INSTRUCTION: U, THE UNPROTECTED IMAGE; P, THE PROTECTED IMAGE; ISA, PROVIDED BY THE COPROCESSOR EXTENSION BUT ISSUED BY NEITHER IMAGE, WHICH COMPUTES THAT STEP IN SOFTWARE. [†]INSTRUCTION 9 DOES NOT COMPUTE THE REDUCTION ITS NAME STATES (SECTION IV-A1).

# Instruction	f3,f7	Algorithm step and secret	Label Z	FW
1 MONT_K	1,1	$\hat{s} \circ \hat{u}$, secret coefficient	HW(s)	P
2 BARRETT_K	1,0	reduction of $v = c_2 - t$	HW(x)	U,P
3 AND_LATCH	6,6	Boolean step of <code>poly_tomsg</code>	HW(w)	P
4 CADDQ	1,2	conditional add of q_K on the sign	$[x < 0]$	U,P
5 NTT_BFLY_GS_MUL	2,3	inverse butterfly on t	HW($a+b$)	ISA
6 COMPRESS_1	4,0	message bit from v	bit	ISA
7 CBD2_1	3,0	noise of the Fujisaki–Okamoto re-encryption	coeff.	P
8 BARRETT_D	1,3	reduction of $z = y + cs_1$	HW(x)	U,P
9 MONT_D_REDUCE64 [†]	1,4	product cs_1 , 64-bit	$s \in [-2, 2]$	ISA
10 DECOMPOSE	5,1	(w_1, w_0) from $w = Ay$	HW(r_0)	ISA

IV. PRE-SILICON AND POST-SILICON ANALYSIS

A. Selecting the secret-carrying instructions

The analysis quantifies, per secret-carrying instruction, how much a power trace reveals about that secret. It predicts leakage from RTL while the design can change, and it verifies the prediction on the board. The coprocessor offers 62 instructions, and an evaluation must choose where to spend its traces. Disassembly of the linked images shows uneven use: unprotected ML-KEM issues ten distinct encodings over 38 sites, only two on a secret coefficient, protected ML-KEM issues 27 encodings over 113 sites, unprotected ML-DSA issues one, `BARRETT_D`, and protected ML-DSA issues nine, the generator seeding and read-out and `BARRETT_D`. Firmware use bounds a study of these images, but it does not bound the interface, and the selection therefore follows the algorithms rather than the call frequency of one firmware.

An instruction is in scope when the ISA offers it for a step that handles a secret, whatever the present firmware calls. Table III lists the ten with the label Z of the earlier specific test and the firmware that issues each, and Listing 1 places them in the two algorithms. Seven belong to ML-KEM-1024 decapsulation and three to ML-DSA-44 signing, and together they exercise six RTL units, and a result on one unit is never carried to another. The Keccak group runs on hash state, the masking group carries shares, and the remaining sampling and compression widths share logic with a listed row. The selection is manual and maps algorithm steps to instructions without taint analysis, which leaves an omitted instruction unjudged rather than cleared. Three of the ten merit detail, because each shows one thing the assessment must handle.

1	$\hat{u} \leftarrow \text{NTT}(\text{Decompress}(c_1))$	
2	$\hat{t} \leftarrow \hat{s} \circ \hat{u}$	MONT_K
3	$t \leftarrow \text{NTT}^{-1}(\hat{t})$	NTT_BFLY_GS_MUL
4	$v \leftarrow \text{Decompress}(c_2) - t \bmod q_K$	BARRETT_K, CADDQ
5	$m' \leftarrow \text{ByteEnc}_1(\text{Compress}_1(v))$	COMPRESS_1, AND_LATCH
6	$(K', r') \leftarrow G(m' \ h)$	
7	$c' \leftarrow \text{K-PKE.Encrypt}(ek, m', r')$	CBD2_1
8	return K' if $c' = c$, else the rejection key	

1	sample the nonce y	
2	$w \leftarrow Ay$	
3	$(w_1, w_0) \leftarrow \text{Decompose}(w)$	DECOMPOSE
4	$c \leftarrow H(\mu \ w_1)$	
5	$p \leftarrow c s_1$ in the NTT domain	MONT_D_REDUCE64
6	$z \leftarrow y + p$	
7	reduce the coefficients of z	BARRETT_D
8	restart if $\ z\ _\infty$ or the hint exceeds its bound	

Listing 1: Where the ten instructions act. Above, ML-KEM-1024 decapsulation with seven of them; below, ML-DSA-44 signing with the other three. Each annotated step handles the secret key $\hat{s}$ or s_1 , or a value derived from it.

1) *Three instructions in detail:* CADDQ receives a signed secret-derived coefficient and adds q_K when its sign bit is set. This addition is the final reduction step, and it yields the value v that message decoding consumes. The label is the sign bit, on which the datapath selects one of two values, and the switching happens as the operand arrives. CBD2_1 starts centred-binomial sampling with $\eta = 2$. Because the Fujisaki–Okamoto re-encryption seeds this sampling from the recovered message, the output coefficient in $[-2, 2]$ derives from the secret, and that five-valued coefficient, returned over the shared 32-bit port, is the label.

MONT_D_REDUCE64 receives the 64-bit product of a public factor c and a secret coefficient s_1 . A Montgomery reduction of a 64-bit input a computes

$$t = \text{int}32(a q_D^{-1} \bmod 2^{32}), \quad (1)$$

$$r = (a - t q_D) / 2^{32}, \quad (2)$$

and a correct unit satisfies $r 2^{32} \equiv a \pmod{q_D}$ with $-q_D < r < q_D$ for inputs in the product range of the signing multiplication. The original unit uses $-q_D^{-1}$ where the subtractive form needs $+q_D^{-1}$, and it satisfies the identity for only 3 of 10,011 signed test inputs. A corrected unit with $q_D^{-1} \bmod 2^{32} = 58728449$ and a signed low-word multiplier passes identity, reference equality and range check on all 10,011, but every measurement here uses the original bitstream, and the unit stays in the ten because it receives secret data, whatever it computes. Standard signing evaluates

$$c s_1 = \text{NTT}^{-1}(\text{NTT}(c) \circ \text{NTT}(s_1)), \quad (3)$$

where the transformed coefficients of s_1 are not restricted to $[-2, 2]$, and the label here is a synthetic $s \in [-2, 2]$ times a full-range factor and its row is a diagnostic rather than an ML-DSA recovery result. The other three reduction units pass their identity on 5000 of 5000 inputs, and the six units without one pass the range their FIPS step defines.

B. Pre-silicon setup

Verilator simulates each instruction in the full system on the board firmware, compiled one instruction at a time, and each instruction keeps its loop and pipeline context. Because the simulated system runs the firmware of the board, each isolated instruction executes as it does on silicon. The power proxy is the toggle count, the number of signal transitions from zero to one or one to zero in the whole design during one clock cycle. It stands in for dynamic power, since CMOS switching energy scales with those transitions, and it weights every changed bit equally.

One simulated trace stores twelve such counts, one per cycle from the CV-X-IF issue handshake. Cycles 0 and 1 deliver the operands across the interface, cycles 2 to 5 cover the coprocessor datapath, and cycles 6 to 11 cover write-back through the four-stage core pipeline and the drain, and Section V shows that six instructions peak in cycle 1 or 2 and two in cycle 8. The operand changes on every random run, and 1000 runs per instruction take about one minute. This zero-delay RTL model omits glitches, routing capacitance and coupling, which gives a noise-free baseline that Section V-D calibrates against measured noise.

C. Post-silicon setup

The target is a CW305 board with an XC7A100T FPGA clocked at 16 MHz from the ChipWhisperer [24]. A ChipWhisperer-Lite samples the VCCINT shunt at 64 MHz, four samples per cycle, over a 500-sample window that a TIO4 trigger opens around one instruction, and the twelve simulated cycles therefore map to 48 samples. The firmware sequences itself and publishes the block and operand index in a mailbox after each capture, which gives every trace its own label.

The capture gain follows from measurement. At 20 dB the signal-to-noise at the leaking sample reaches +1.3 dB but 14 of 150 traces hit the converter rail, at 15 dB it reaches +2.5 dB with no clipped trace among 20,000, and at 12 dB it drops to -2.2 dB. The campaign therefore uses 15 dB. Two passes fifteen minutes apart each capture 1000 traces per instruction on identical operands, with the ten instructions interleaved so that slow drift cannot align with one instruction, and the second pass supplies the replication test of Section V-D.

D. Leakage test in each domain and correlation between them

Each instruction runs 1000 times per domain, 500 with a fixed operand and 500 with a random operand. Because the six single-operand instructions carry the secret in their one operand, that operand is fixed in one class and random in the other, the fixed-versus-random key test of TVLA [25]. The four two-operand instructions hold the secret at one non-zero constant in every run, and their public operand is fixed in one class and random in the other, the classical fixed-versus-random data test. Consecutive runs form pairs, and each pair holds one fixed and one random run in random order, which keeps the class independent of the loop index and of slow drift. An unmeasured run of the same instruction on one constant operand precedes every measured run outside the window,

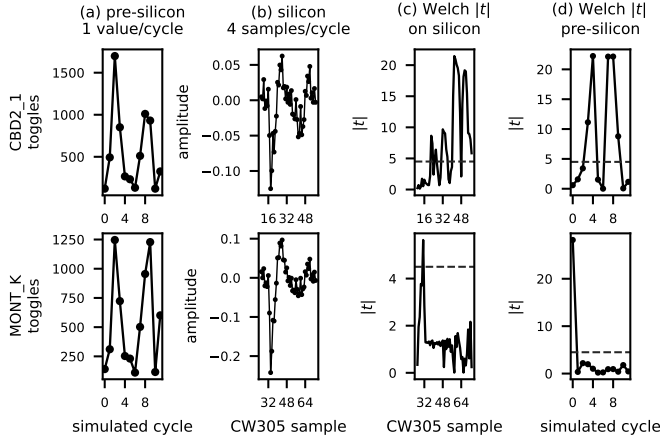


Fig. 4. One execution of the strongest and one of the weakest instructions in each domain, from the earlier label-split capture and not the fixed-versus-random peaks of Table IV. Column (a) is the simulated toggle count, one value per cycle; column (b) the measured trace, four samples per cycle; column (c) Welch’s t of a median split of the label on the board; column (d) the same split in simulation. On the board CBD2_1 exceeds 4.5 at 26 of its 48 samples and peaks at 21.5, MONT_K at one sample with a peak of 5.6. In simulation they peak at 22.2 and 28.2.

and every measured run therefore has the same predecessor operand. Fig. 4 shows both domains on one axis for the strongest and one of the weakest instructions.

Welch’s t compares the two classes at every cycle in simulation and at every sample on the board. A cycle or sample whose statistic exceeds 4.5 in magnitude marks leakage, while the largest magnitude is the peak of the instruction, and the second board pass repeats the test on the same operands and is scored separately. Four controls accompany every peak. The random class split in two halves must stay below 4.5, the class labels shuffled 100 times must stay below the observed peak, the fixed class split by the class of the preceding measured run measures the history the constant run leaves, and the fixed rows that follow a random run against the random rows repeat the test with matched history. Where a single-trace attack is evaluated, a Gaussian template is fitted on a profiling half and the rank of the true secret is scored on the other half.

The two domains meet only through correlation. Let $S_{n,c}$ be the simulated toggle count of trace n at cycle c and $A_{n,j}$ its measured amplitude at sample j ; the grid

$$C_{c,j} = \text{corr}_n(S_{n,c}, A_{n,j}) \quad (4)$$

spans 12 cycles by 500 samples over the 500 random-class traces paired by operand, and its largest magnitude is the cross-domain statistic, judged against the largest magnitude over 100 re-pairings of the same traces. That statistic measures shared data-dependent activity, since the grid maximum is selected on the same traces and no secret-bearing intermediate is named. No t value crosses the domains, because a noise-free model gives every fixed run one count and its statistic scales with the distance of one fixed value from the mean, and at 500 runs per class Cohen’s d equals $t/15.8$ in either domain.

TABLE IV
FIXED-VERSUS-RANDOM TEST, 500 RUNS PER CLASS: PEAK WELCH’S t PER DOMAIN, SECOND BOARD PASS, AND CROSS-DOMAIN $|r|$ AGAINST ITS RE-PAIRING NULL.

# Instruction	$ t $ sim	$ t $ CW305	$ t $ pass 2	$ r $	null
Secret operand fixed against random					
2 BARRETT_K	45.7	41.9	41.9	0.795	0.21
4 CADDQ	24.4	20.2	20.6	0.978	0.21
6 COMPRESS_1	20.9	13.6	13.3	0.776	0.21
7 CBD2_1	16.8	19.6	20.0	0.992	0.21
8 BARRETT_D	5.5	19.9	18.6	0.760	0.21
10 DECOMPOSE	6.4	18.8	20.2	0.819	0.21
Secret constant, public operand fixed against random					
1 MONT_K	26.2	38.5	36.8	0.888	0.21
3 AND_LATCH	32.4	7.7	8.1	0.467	0.23
5 NTT_BFLY_GS_MUL	33.2	25.0	30.1	0.765	0.21
9 MONT_D_REDUCE64	33.0	26.3	25.6	0.995	0.21

V. RESULTS AND DISCUSSION

A. Unprotected instructions leak in both domains

Every instruction crosses 4.5 in simulation and in both board passes, and every peak exceeds the largest peak of 100 label shuffles, 5.3 over the 500 searched samples. Table IV gives the peaks, the board peak runs from 7.7 for AND_LATCH to 41.9 for BARRETT_K, and the second pass repeats each peak within 1.7 on nine instructions while NTT_BFLY_GS_MUL moves from 25.0 to 30.1. The six single-operand rows vary the secret, and their crossing shows that power depends on its value. Because the four two-operand rows hold the secret constant, their crossing shows dependence on the public operand processed under it, and neither test names the leaking intermediate.

Table VIII in Appendix A lists the controls per instruction. The random class split in halves reads at most 4.3, the fixed class split by the class of the preceding measured run at most 4.2, and the fixed rows that follow a random run repeat every peak within 1, which places the crossing on the operand value rather than on history. BARRETT_D and DECOMPOSE peak at 5.5 and 6.4 in simulation against 19.9 and 18.8 on the board, because the toggle model omits glitches and routing capacitance and each instruction has one fixed value. MONT_D_REDUCE64 stays above 4.5 for 200 samples after its window, because the firmware consumes its result.

Detection differs from recovery. An earlier capture on its own operand tables scored each instruction against its label of Table III on a median split, and eight of ten cross within 100 simulated and 1000 measured traces, while COMPRESS_1 and DECOMPOSE stay below 4.5 on that label in both domains, which detects no dependence on the bit that decoding consumes at that trace count while the coefficient value crosses. A template profiled on half of that capture ranks the true MONT_K coefficient first in 1 of 400 traces among 3329 candidates, although MONT_K crosses on its label at 500 traces. The five-valued operand of MONT_D_REDUCE64 ranks first from one trace in 55.0 % of 400 traces on the board, a 95 % interval of 50 to 60 %, and 75.8 % in simulation against a chance rate of 20 %, and the diagnostic uses operands the adversary of Section II lacks.

TABLE V

PROTECTED FORMS FROM THE EARLIER SIX-BLOCK CAPTURE (XC7A100T, 20 dB) AND THE SHARE-RESIDENT GADGET: TRACES N , CORRELATION STATISTIC OF THE HAMMING WEIGHT OF THE SECRET WITHIN THE INSTRUCTION, AND AFTER THE FIRMWARE RECOMBINES THE SHARES.

Protected form of	Shares	N	$ t $ secret	recomb.	Verdict
MONT_K (SECMUL)	arith.	872	3.50	–	below
same, two passes	arith.	882 [‡]	3.24	–	below
BARRETT_K	arith.	999	1.80 [†]	7.42	below
AND_LATCH (DOM AND)	Boolean	851	1.86 [†]	7.83	below
Resident DOM χ (35T) [§]	Boolean	85k	5.40	–	crosses

[†]recombination at sample 41 and 89. [‡]882 operands, each averaged from its raw traces, 1759 in total. [§]CROC sibling, XC7A35T, fixed-versus-random, traces per class; 4.3 at 50k and 5.40 at 85k against 6.71 with the PRNG off.

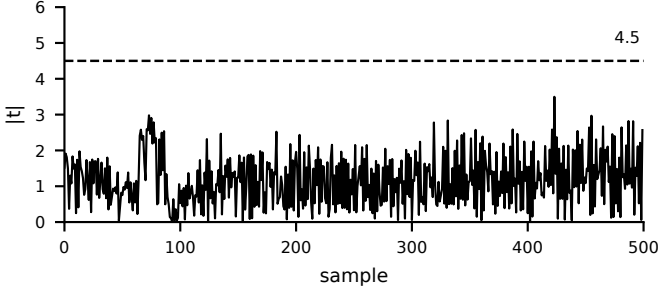


Fig. 5. Masked multiplier that replaces MONT_K, 872 CW305 traces of the six-block capture: statistic of the unmasked secret at every sample.

Leakage follows mechanism, not RTL unit. CADDQ selects an addition on the sign bit of its operand, and the switching follows that selection. The reduction units and CBD2_1 return their result over one 32-bit port, whose simulated toggle count for BARRETT_K rises by 14.02 per bit of Hamming weight at write-back.

B. Protected forms

Fig. 5 shows the masked multiplier that replaces MONT_K on 872 traces of the six-block capture, scored with the correlation statistic of the Hamming weight of the unmasked secret rather than the fixed-versus-random test. The statistic stays below 4.5 at every sample, and it peaks at 3.50. This test detects no first-order dependence on the secret at 872 traces, which is a statement about this test and trace count rather than a proof of absence.

Table V lists the protected forms of that capture. Because it ran at 20 dB with another bitstream of the same design, only its verdicts carry over. The masked multiplier stays below 4.5 on 1759 raw traces averaged into 882 operands, its single-share control fires at 9.84, and the resident DOM χ gadget on the CROC sibling, a second SoC that carries the same coprocessor on an XC7A35T, stays below 4.5 to 50,000 traces per class but crosses at 85,000 with 5.40, against 6.71 with its generator off. That gadget therefore fails the test at that trace count on the sibling design, and the failure is unresolved here. The masked reduction and the masked AND rise to 7.42 and 7.83 only where the test firmware adds or exclusive-ors the two result shares in the core, and before that point the secret statistic peaks at 1.80 and 1.86.

C. Agreement between the domains

The two domains agree on every instruction. The last column of Table IV lists the largest grid cell on the random-class traces, and every instruction exceeds its re-pairing null, at most 0.23, from 0.47 for AND_LATCH to 0.995 for MONT_D_REDUCE64. All ten correlations are negative, which points to one measurement polarity, and the three largest belong to the instructions with the fewest operand levels, which makes the column a measure of shared activity rather than of a common secret-bearing intermediate. AND_LATCH exclusive-ors a fresh generator word into its result on every issue, and its result carries little even with unmasked operands. The verdict agrees on all ten, which a uniform verdict cannot discriminate, while the peak magnitudes rank at a Spearman coefficient of 0.56, with AND_LATCH at 32.4 simulated against 7.7 measured. The largest correlation of a column with the acquisition index is 0.320 in simulation and 0.555 on the board, a diagnostic that the paired class order keeps out of the test.

The simulated peak cycle locates the switching in the RTL. MONT_K and DECOMPOSE peak in cycle 1 at operand delivery, and BARRETT_K, AND_LATCH, NTT_BFLY_GS_MUL and BARRETT_D in cycle 2, the first datapath cycle. CBD2_1 and CADDQ peak in cycles 3 and 4, while COMPRESS_1 and MONT_D_REDUCE64 peak in cycle 8, where the result returns, and Appendix A shows the measured statistic per sample.

D. Noise on the board

The earlier capture also measured the noise at the leaking sample of every instruction, and it spans nearly forty decibels, from -20.5 dB for COMPRESS_1 to $+18.5$ dB for CBD2_1, on one board, one gain and one window. Reading the simulation at each instruction's measured noise brings the information one trace gives about its label to a mean absolute error of 0.056 bits per trace against the board, at most 0.21 for CADDQ, whereas the noise-free count misses by 0.666 bits. The noise comes from the board, and the result is a calibrated transfer rather than an independent prediction. A pre-silicon assessment should state effect size at a stated noise level.

Two passes over identical operands test the usual replication argument. Independent noise would double the signal-to-noise of their average, but the measured gain runs from 0.68 for COMPRESS_1 to 1.83 for CBD2_1. A gain below one means the signal itself moved between passes at that sample, and interference from the surrounding program repeats with the operand, which means a replication test needs fresh operands per pass.

E. Performance, area and limits

Table VI gives the cycle counts of both algorithms. The protected build costs 2.2 times the unprotected one on both, while protected ML-KEM decapsulation still runs 18 times faster than software with the software-masked Keccak. ML-DSA-44 signing reaches the coprocessor at one reduction site, and its protected form incurs the 2.2 times without a speed-up

TABLE VI
CYCLE COUNTS FROM THE `MCYCLE` COUNTER ON THE BOARD AT 16 MHZ, SOFTWARE AGAINST THE UNPROTECTED (U) AND PROTECTED (P) BUILDS.

Operation	Build	Cycles (k)	Speed-up	Cost vs U
ML-KEM-1024 decapsulation	software	7,402	1×	–
	U	180	41×	1×
	P	401	18×	2.2×
ML-DSA-44 signing	U	380	–	1×
	P	820	–	2.2×

TABLE VII
WALL-CLOCK TIME PER STEP FOR TEN INSTRUCTIONS AT 1000 RUNS EACH, ONE WORKSTATION AND ONE CW305.

Step	Pre-silicon	Post-silicon
Build	firmware, seconds	bitstream, 15 min
Capture, ten instructions	15 min	20 min per pass
Analysis, all tests	<3 min	<3 min

to offset it. Table VII gives the time of each step, and the bitstream is the one cost the board adds beyond the capture.

Table II gives the measured build on the XC7A100T at 53.7 % of LUT halves without the hardware masked Keccak. On the XC7A35T the 38-instruction coprocessor adds 14.7 % LUT, 10.2 % flip-flops and 48 DSP48E1 over the software-only SoC, and the protected form adds nothing, because both run on one netlist [23]. With the resident masked Keccak the design reaches 92,020 LUT halves in the synthesis report, 73 % of the part, before the placer fails. The campaign has five limits, one board, one device, one placement, one fixed value per instruction, and protected forms measured in the earlier 20 dB capture only, for which a recapture at 15 dB with firmware that never recombines the shares is the next step. `BARRETT_K` runs on operands already in $[0, q_K)$, which measures the unit’s transport cost rather than its reduction, and the ten run in isolation rather than inside a decapsulation or signature.

The verdicts hold per instruction, and they do not compose to the algorithm. An instruction that passes alone can leak in sequence, because a result left in a register or a port meets the next operand, or because the randomness of one masked instruction is reused by the next. A composability test, the same fixed-versus-random test over an instruction sequence with the intermediate state and the mask supply of the real firmware, is the natural next step once each instruction has passed on its own.

VI. CONCLUSION

This study tests a pre-silicon leakage assessment of a PQC coprocessor against silicon. The design is open source for both lattice standards with and without a first-order countermeasure, and an open flow carries it from SystemVerilog to a configured CW305. When ten selected secret-relevant instructions are assessed in both domains over identical operands, the toggle counts and the measurements agree on every detection verdict, while their peak magnitudes differ and their activity correlates at a mean of 0.824.

Every unprotected instruction crosses 4.5 within 500 runs per class in both domains, with secret dependence shown on six and public-operand dependence on four. The three measured protected forms show no detectable leakage within their instruction at this trace count, and their only crossings sit where the test firmware recombined the result shares. Each verdict covers one instruction, and composition across instructions is the next test. Because measurement noise decides what a pre-silicon verdict is worth, the method states each verdict at the noise its instruction shows on the board, while the design can still change.

ACKNOWLEDGEMENT

The authors thank the developers of Yosys, nextpnr-xilinx, Project X-Ray, Verilator, CV32E40X and ChipWhisperer, whose open tools made an inspectable path from source to measurement possible. Generative AI was used to polish the language of parts of this manuscript. The authors reviewed all AI-assisted text and are responsible for the final content.

REFERENCES

- [1] National Institute of Standards and Technology, “FIPS 203: Module-lattice-based key-encapsulation mechanism standard,” Federal Information Processing Standards Publication, Aug. 2024.
- [2] National Institute of Standards and Technology, “FIPS 204: Module-lattice-based digital signature standard,” Federal Information Processing Standards Publication, Aug. 2024.
- [3] National Institute of Standards and Technology, “Transition to post-quantum cryptography standards,” NIST IR 8547, initial public draft, Nov. 2024.
- [4] N. Müller, D. Lammers, S. Osterheider, and A. Moradi, “Coupling leakage in theory and practice: Unveiling (post-PnR) security flaws in masked FPGA-mapped designs,” IACR Cryptology ePrint Archive, Paper 2026/1426, 2026.
- [5] P. Schaumont, “You can hide but you can’t verify: On side-channel countermeasure verification,” in *Proc. Workshop on Safety and Security in Heterogeneous Open System-on-Chip Platforms (SSH-SoC), co-located with DAC*, 2023.
- [6] ISO/IEC, “ISO/IEC 17825:2024, information technology, security techniques, testing methods for the mitigation of non-invasive attack classes against cryptographic modules,” 2024.
- [7] M.-J. O. Saarinen, “WiP: Applicability of ISO standard side-channel leakage tests to NIST post-quantum cryptography,” in *Proc. IEEE Int. Symp. Hardware Oriented Security and Trust (HOST)*, 2022.
- [8] P. Kiaei, Z. Liu, R. K. Eren, Y. Yao, and P. Schaumont, “Saidoyoki: Evaluating side-channel leakage in pre- and post-silicon setting,” IACR Cryptology ePrint Archive, Paper 2021/1235, 2021.
- [9] Z. Liu, A. Malnicof, A. Roy, and P. Schaumont, “Telescope: Top-down hierarchical pre-silicon side-channel leakage assessment in system-on-chip design,” in *Proc. 20th ACM Asia Conf. Computer and Communications Security (ASIA CCS)*, 2025, pp. 1280–1293.
- [10] A. Dolmeta, V. Piscopo, G. Masera, M. Martina, and M. Hutter, “HOR-CRUX: A lightweight PQC-RISC-V extension architecture,” IACR Cryptology ePrint Archive, Paper 2025/1934, 2025.
- [11] E. Camacho-Ruiz, P. Navarro-Torrero, and A. Cabrera Aldaya, “A framework for designing high-order side-channel protected hardware implementations of ML-KEM,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, vol. 2026, no. 2, pp. 272–295, 2026.
- [12] P. Dobias, A. Aikata, A. Rezaeezade, L. Batina, L. Malina, and T. Gerlich, “Two birds, one mask: Side-channel-resistant unified hardware for ML-DSA and ML-KEM,” *PeerJ Computer Science*, vol. 12, p. e4003, 2026.
- [13] A. Adhikary, A. J. Basurto Becerra, L. Batina, I. Buhan, D. Chatterjee, S. van Hoek, and E. Sanfelix Gonzalez, “ARCHER: Architecture-level simulator for side-channel analysis in RISC-V processors,” IACR Cryptology ePrint Archive, Paper 2024/1866, 2024.

- [14] A. Nešković, C. Ewert, M. Berekovic, and S. Mulhem, “SPARC: Automated root-cause analysis of pre-silicon power side-channel leakage in the processor design flow,” arXiv:2607.23218, to appear in Proc. IEEE/ACM ICCAD, 2026.
- [15] B. Farnaghinejad, A. Porsia, A. Ruospo, A. Savino, S. Di Carlo, and E. Sanchez, “Power side-channel analysis of the CVA6 RISC-V core at the RTL level using VeriSide,” arXiv:2512.21362, 2025.
- [16] T. Fritzmann, M. Van Beirendonck, D. Basu Roy, P. Karl, T. Schamberger, I. Verbauwhe, and G. Sigl, “Masked accelerators and instruction set extensions for post-quantum cryptography,” *IACR Trans. Cryptographic Hardware and Embedded Systems*, vol. 2022, no. 1, pp. 414–460, 2022.
- [17] B. Gigerl, V. Hadzic, R. Primas, S. Mangard, and R. Bloem, “Coco: Co-design and co-verification of masked software implementations on CPUs,” in *Proc. 30th USENIX Security Symposium*, 2021, pp. 1469–1486.
- [18] P. Sauter, T. Benz, P. Scheffler, F. K. Gürkaynak, and L. Benini, “Insights from Basilisk: Are open-source EDA tools ready for a multi-million-gate, Linux-booting RV64 SoC design?” in *Proc. 33rd Int. Workshop on Logic and Synthesis (IWLS)*, 2024, arXiv:2405.04257.
- [19] OpenHW Group, “CV32E40X RISC-V core and the CORE-V extension interface (CV-X-IF),” <https://github.com/openhwgroup/cv32e40x>, 2024.
- [20] H. Groß, S. Mangard, and T. Korak, “Domain-oriented masking: Compact masked hardware implementations with arbitrary protection order,” in *Proc. ACM Workshop on Theory of Implementation Security (TIS@CCS)*, 2016, p. 3.
- [21] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanović, “Yosys+nextpnr: An open source framework from Verilog to bitstream for commercial FPGAs,” in *Proc. IEEE 27th Int. Symp. Field-Programmable Custom Computing Machines (FCCM)*, 2019, pp. 1–4.
- [22] F4PGA project, “Project X-Ray: Documenting the Xilinx 7-series bitstream format,” <https://github.com/f4pga/prjxray>, 2024.
- [23] D. Shanmugam and P. Schaumont, “Open-source reference for reproducible side-channel assessment of post-quantum standards,” in *Proc. Workshop on Open-Source Computer Architecture Research (OSCAR)*, co-located with ISCA, 2026.
- [24] NewAE Technology Inc., “CW305 Artix-7 FPGA target and ChipWhisperer-Lite capture hardware,” <https://www.newae.com>, 2024.
- [25] G. Goodwill, B. Jun, J. Jaffe, and P. Rohatgi, “A testing methodology for side-channel resistance validation,” in *NIST Non-Invasive Attack Testing Workshop (NIAT)*, 2011.

APPENDIX A

MEASURED STATISTIC PER SAMPLE

Fig. 6 shows Welch’s t of the fixed-versus-random test at every CW305 sample of the first pass for the ten instructions. Each peak sits inside the first 66 samples, which cover the twelve simulated cycles and the trigger offset at four samples per cycle. Table VIII lists the four controls of Section IV-D per instruction and domain: the peak, the fixed rows that follow a random run against the random rows (FaR), the fixed rows split by the class of the preceding measured run (FF), the random class split in halves (RR), and the largest peak of 100 label shuffles (null).

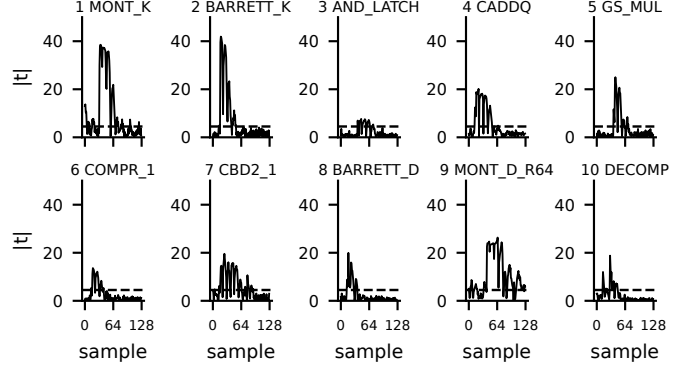


Fig. 6. Welch’s t of the fixed-versus-random test per CW305 sample on the first pass, 500 runs per class, first 128 samples of each window against the dashed 4.5 line.

TABLE VIII
CONTROLS PER INSTRUCTION, SIMULATION AND CW305 PASS 1:
WELCH’S t OF THE FIXED-VERSUS-RANDOM TEST (PEAK) AND OF ITS
FOUR CONTROLS.

# Instruction	simulation					CW305, pass 1				
	peak	FaR	FF	RR	null	peak	FaR	FF	RR	null
1 MONT_K	26.2	26.7	5.3	1.2	3.2	38.5	38.5	1.1	2.4	4.7
2 BARRETT_K	45.7	45.7	3.8	1.2	3.3	41.9	41.7	2.0	2.9	4.0
3 AND_LATCH	32.4	33.1	5.7	1.3	3.5	7.7	7.8	1.4	2.1	4.4
4 CADDQ	24.4	24.2	16.8	2.6	3.5	20.2	20.5	4.2	2.9	4.2
5 NTT_BFLY_GS_MUL	33.2	33.3	20.9	1.5	2.8	25.0	25.0	1.6	3.5	4.0
6 COMPRESS_1	20.9	19.7	17.4	2.6	3.3	13.6	13.6	0.1	2.6	4.0
7 CBD2_1	16.8	16.6	1.7	2.1	3.4	19.6	19.6	1.0	2.0	4.1
8 BARRETT_D	5.5	5.4	21.4	2.2	3.3	19.9	19.8	1.5	2.6	4.0
9 MONT_D_REDUCE64	33.0	33.0	0.0	2.4	2.8	26.3	26.2	2.9	3.2	4.7
10 DECOMPOSE	6.4	6.8	5.1	1.8	3.5	18.8	17.8	0.1	3.1	4.7